CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 06

Semaphores



Semaphores

Topics to be Covered

1. Semaphores
2. Monitors

Introduction to Semaphores

- **Semaphore** is an **operating system tool** used for **process synchronization**.
- It helps to control access to shared resources by multiple processes in a concurrent system.

Types of Semaphores

1. Binary Semaphore
2. Counting Semaphore

1. Binary Semaphore

- A **binary semaphore** is an **integer variable** accessed by cooperating processes using two operations:
  - wait()
  - signal()
- It is **initialized to 1** by the OS and can hold only **two values: 1 or 0**.
  int s = 1; /* Let s be a binary semaphore initialized to 1 */

Wait Operation

- A process performs the wait() operation before entering the **critical section**.

Working:

- The first process that performs wait() will set the semaphore to **0** and **enter** the critical section.
- If another process (say P_1) also tries to enter when one process is already inside, it will **wait** until the semaphore value becomes **1** again.

```
void wait(int *S) {
   while (*S == 0)  /* keep looping as long as S = 0 */
      ;          /* busy waiting */
   *S = *S - 1;
}
```

Note:

- Only one waiting process can enter at a time.
- The process keeps checking the value of the semaphore in a loop (busy waiting).

Signal Operation

- The signal() operation is used when a process **exits** its critical section.
- It **increments** the semaphore value by 1, allowing one of the waiting processes to proceed.

Explanation: When a process completes its critical section, it performs a **signal()** to **wake up another waiting process**.

Binary semaphores are also called spin locks.

Properties:

- Satisfies **Mutual Exclusion**
- Satisfies **Progress**
- Does **not** satisfy **Bounded Waiting**

Limitations of Binary Semaphores

1. **Random selection of processes:**
   - The process chosen to enter next is random, so **some processes may repeatedly enter** while others may starve forever.
   - Hence, the **Bounded Waiting** requirement is not met.
2. **Busy Waiting (Spinlock):**
   - Waiting processes **continuously check** the semaphore value, wasting CPU cycles.
   - This is why binary semaphores are also called **spinlocks**.

Advantages and Counting Semaphore Introduction

Advantage of Binary Semaphore

- **Very simple to implement**.

2. Counting Semaphore

- Counting semaphores **overcome the limitations** of binary semaphores.
- They can take **any non-negative integer value** and are used when multiple instances of a resource exist.

Interpretation of Count Values:

- **Semaphore Count = 1:** No process is executing in its critical section, and no process is waiting.
- **Semaphore Count = 0:** One process is executing in the critical section, and no process is waiting.
- **Semaphore Count = N:** One process is in the critical section, and **N processes are waiting** in the semaphore queue.
- A waiting process **does not perform busy waiting**. Instead, it moves to one of the following states:
  - **Blocked State**
  - **Binary/Waiting State**

State Changes:

- When a waiting process is selected for entry into the critical section:
  - It moves from **Blocked → Ready State**.
  - Then, it enters the **critical section** for execution.

Advantages of Counting Semaphore (Over Binary Semaphore)

(a) Waiting processes enter the critical section in **FCFS (First-Come, First-Served)** order, satisfying **Bounded Waiting**.

(b) Waiting processes **do not perform busy waiting**, thus **saving CPU cycles**.

Disadvantages of Counting Semaphore

(a) Complex Implementation:

- Needs an **FCFS queue**, making implementation more complex.

(b) Extra Context Switching:

- When a process cannot enter its critical section:
  1. It moves from **Running → Waiting State**.
  2. Later, from **Waiting → Ready State**.
- These extra context switches introduce **overhead**.

Classical Synchronization Problems

Synchronization problems that use semaphores for solutions:

1. The Bounded Buffer (Producer–Consumer) Problem

- A **producer** generates data items that are consumed by a **consumer**.
- The producer and consumer work **asynchronously**.
- Sometimes, the producer may produce faster than the consumer can consume.
- A **bounded (limited) buffer** temporarily holds produced items until they are consumed.
- Communication between producer and consumer is done using **shared memory** and **semaphores**.

Dining Philosopher's Problem (Part 1)

- A **classical synchronization problem** used to illustrate process coordination.

Description:

a. N philosophers are sitting around a circular table.
b. There are **N plates**, one in front of each philosopher.
c. There are **N chopsticks**, placed between each pair of plates.
d. There is a **bowl of spaghetti** in the center of the table.
e. When a philosopher feels hungry, they try to **pick up two chopsticks** (shared with their neighbors).
- If a neighbor is already eating, the philosopher must **wait**.

f. Once they acquire both chopsticks, they **eat**.
g. After eating, they **put down both chopsticks** and resume **thinking**.
- The chopsticks then become available to neighbors.

Limitation of Algorithm

- All philosophers might get hungry simultaneously and pick up their **left chopstick** first.
- Each will then wait indefinitely for their **right chopstick**, causing **deadlock**.
- **Progress requirement fails** here.

Deadlock-Free Solution

- Modify the algorithm:
  - **Even-numbered philosophers (0, 2, 4, …)** pick **left** first, then **right**.
  - **Odd-numbered philosophers (1, 3, 5, …)** pick **right** first, then **left**.
- This prevents circular waiting, hence **avoiding deadlock**.

Sleeping Barber Problem

Problem Description

a. A barbershop has **n chairs** in the waiting area and **1 chair** in the barber's cabin.
b. If **no customers** are present, the barber **goes to sleep**.
c. When a customer arrives:
- If the barber is sleeping, the customer **wakes him up** and gets a haircut.

d. If a customer is already getting a haircut:
- The new customer checks for an **empty chair** in the waiting room.
- If available, the customer **sits and waits** (FCFS order).
- If all chairs are occupied, the customer **leaves** without a haircut.

e. When a customer finishes, they **signal** the next waiting customer to enter the barber's cabin and then **leave the salon**.

Readers and Writers Problem

Problem Statement

a. Several cooperating processes are accessing a shared file or database.
b. Some processes only **read** data — called **Readers**.
c. Some processes **modify** data — called **Writers**.
d. **When a Writer** is writing, **no other Reader or Writer** can access the data.
e. **When Readers** are reading, multiple Readers can access simultaneously (no Writer allowed).

Goal:

- Ensure synchronization so that Writers get exclusive access, and multiple Readers can work together safely.

Programming Error 1
Violation of Mutual Exclusion

- Example of incorrect semaphore use:

```
P1:
Signal(&S);  /* Error */
<Critical Section>
```

- If signal() is called **before** entering the critical section, **two processes** may enter simultaneously.
- This **violates mutual exclusion**.

Programming Error 2
Blocking All Processes

- Example of another common error:

```
P2:
Wait(&S);  /* Error */
<Critical Section>
/* signal(&S) is missing */
<Remainder Section>
```

- A missing **signal()** causes all other processes to **wait indefinitely**, blocking progress.

Conclusion:

- Semaphore misuse can lead to **deadlocks** or **mutual exclusion failure**.

Monitors

- **A monitor** is a **high-level synchronization construct** that overcomes the **limitations of semaphores**.
- In semaphores, the operations (wait, signal) are rigid and error-prone.
- **Monitors** encapsulate:
  - **Global variables** are shared among processes.
  - **Procedures/functions** that access these variables.

Key Features:

- Global variables inside a monitor can only be accessed by its own procedures.
- OS ensures:
  - **Mutual Exclusion**
  - **Bounded Waiting**
- Only **one process** can be active inside a monitor at any given time.

Summary of Semaphores

Concept	**Description / Key Points**
Semaphore	Synchronization tool controlling access to shared resources.
Binary Semaphore	Values 0/1; simple but causes busy waiting; called spinlock.
Counting Semaphore	Allows multiple instances of resources; avoids busy waiting.
Advantages of Counting Semaphore	FCFS access, bounded waiting, no CPU wastage.
Disadvantages	Complex implementation; extra context switches.
Classical Problems	Bounded Buffer, Dining Philosopher, Sleeping Barber, Readers–Writers.
Common Errors	Misplaced wait()/signal() causing deadlocks or exclusion failure.
Monitors	High-level synchronization tool ensuring automatic mutual exclusion.

सेमाफोर

कवर किए जाने वाले विषय

1. सेमाफोर
2. मॉनिटर

सेमाफोर का परिचय

- **सेमाफोर** एक **ऑपरेटिंग सिस्टम** टूल है जिसका उपयोग **प्रोसेस सिंक्रोनाइज़ेशन** के लिए किया जाता है।
- यह एक समवर्ती सिस्टम में कई प्रक्रियाओं द्वारा साझा संसाधनों तक पहुंच को नियंत्रित करने में मदद करता है।

सेमाफोर के प्रकार

1. बाइनरी सेमाफोर
2. काउंटिंग सेमाफोर

1. बाइनरी सेमाफोर

- **बाइनरी सेमाफोर** एक पूर्णांक चर है जिसे सहयोगी प्रक्रियाएं **दो संक्रियाओं** का उपयोग करके एक्सेस करती हैं:
  - wait()
  - signal()
- इसे **ऑपरेटिंग सिस्टम** द्वारा 1 से **इनिशियलाइज़** किया जाता है और यह केवल दो मान रख सकता है: 1 या 0।

  int s = 1; /* Let s be a binary semaphore initialized to 1 */

Wait ऑपरेशन

- एक प्रोसेस क्रिटिकल सेक्शन में प्रवेश करने से पहले **wait() ऑपरेशन** करती है।

कार्य:

- **wait()** फ़ंक्शन को निष्पादित करने वाली पहली प्रक्रिया सेमाफोर को 0 पर सेट कर देगी और क्रिटिकल सेक्शन में प्रवेश कर जाएगी।
- यदि कोई अन्य प्रक्रिया (मान लीजिए P_1) भी प्रवेश करने का प्रयास करती है, जबकि एक प्रक्रिया पहले से ही अंदर है, तो वह तब तक प्रतीक्षा करेगी जब तक कि सेमाफोर का मान पुनः 1 न हो जाए।

```
void wait(int *S) {
   while (*S == 0)  /* keep looping as long as S = 0 */
      ;          /* busy waiting */
   *S = *S - 1;
}
```

नोट:

- एक समय में केवल एक ही प्रतीक्षा प्रक्रिया प्रवेश कर सकती है।
- यह प्रक्रिया लूप में सेमाफोर के मान की जाँच करती रहती है (व्यस्त प्रतीक्षा)।

सिग्नल ऑपरेशन

- signal() **ऑपरेशन** का उपयोग तब किया जाता है जब कोई प्रक्रिया अपने क्रिटिकल सेक्शन से बाहर निकलती है।
- यह **सेमाफोर मान** को 1 से बढ़ा देता है, जिससे प्रतीक्षा कर रही प्रक्रियाओं में से एक को आगे बढ़ने की अनुमति मिल जाती है।

स्पष्टीकरण: जब कोई प्रक्रिया अपने क्रिटिकल सेक्शन को पूरा कर लेती है, तो वह किसी अन्य प्रतीक्षारत प्रक्रिया को जगाने के लिए signal() फ़ंक्शन का उपयोग करती है।

बाइनरी सेमाफोर को स्पिन लॉक भी कहा जाता है।

गुणधर्म:

- **पारस्परिक अपवर्जन** को संतुष्ट करता है
- **प्रगति** को संतुष्ट करता है
- **सीमित प्रतीक्षा** को संतुष्ट नहीं करता है

बाइनरी सेमाफोर की सीमाएँ

1. **प्रक्रियाओं का यादृच्छिक चयन:**
    - अगली प्रक्रिया का चयन यादृच्छिक होता है, इसलिए कुछ **प्रक्रियाएँ बार-बार प्रवेश** कर सकती हैं जबकि अन्य हमेशा के लिए प्रतीक्षाहीन रह सकती हैं।
    - इसलिए, **परिबद्ध प्रतीक्षा** की आवश्यकता पूरी नहीं होती है।
2. **व्यस्त प्रतीक्षा (Spinlock):**
    - प्रतीक्षा कर रही **प्रक्रियाएँ लगातार सेमाफोर मान** की जाँच करती रहती हैं, जिससे CPU चक्र बर्बाद होते हैं।
    - इसीलिए बाइनरी सेमाफोर को **स्पिनलॉक** भी कहा जाता है।

सेमाफोर के लाभ और गणना का परिचय

बाइनरी सेमाफोर के लाभ

- इसे लागू करना बहुत आसान है।

काउंटिंग सेमाफोर

- गिनती वाले **सेमाफोर बाइनरी सेमाफोर** की सीमाओं को दूर करते हैं।
- ये कोई भी **गैर-ऋणात्मक पूर्णांक मान** ले सकते हैं और इनका उपयोग तब किया जाता है जब किसी संसाधन के कई उदाहरण मौजूद हों।

2. **गणना मूल्यों की व्याख्या:**
    - **सेमाफोर काउंट = 1:** कोई भी प्रक्रिया अपने क्रिटिकल सेक्शन में नहीं चल रही है, और कोई भी प्रक्रिया प्रतीक्षा नहीं कर रही है।
    - **सेमाफोर काउंट = 0:** एक प्रक्रिया क्रिटिकल सेक्शन में चल रही है, और कोई भी प्रक्रिया प्रतीक्षा नहीं कर रही है।
    - **सेमाफोर काउंट = N:** एक प्रक्रिया क्रिटिकल सेक्शन में है, और N प्रक्रियाएं सेमाफोर कतार में प्रतीक्षा कर रही हैं।
    - प्रतीक्षा कर रही **प्रक्रिया व्यस्त प्रतीक्षा** नहीं करती है। इसके बजाय, यह निम्न में से किसी एक स्थिति में चली जाती है:
        - **अवरुद्ध स्थिति**
        - **द्विआधारी/प्रतीक्षा स्थिति**

राज्य परिवर्तन:

- जब किसी प्रतीक्षारत प्रक्रिया को क्रिटिकल सेक्शन में प्रवेश के लिए चुना जाता है:
    - यह अवरुद्ध अवस्था से तैयार अवस्था में चली जाती है।
    - फिर, यह निष्पादन के लिए क्रिटिकल सेक्शन में प्रवेश करती है।

बाइनरी सेमाफोर की तुलना में काउंटिंग सेमाफोर के फायदे

(a) प्रतीक्षारत प्रक्रियाएं क्रिटिकल सेक्शन में **FCFS (फर्स्ट-कम, फर्स्ट-सर्व्ड) क्रम** में प्रवेश करती हैं, जिससे सीमित प्रतीक्षा का नियम पूरा होता है।

(b) **प्रतीक्षारत प्रक्रियाएं** व्यस्त प्रतीक्षा नहीं करती हैं, जिससे **CPU चक्रों** की बचत होती है।

सेमाफोर की गणना के नुकसान

(a) जटिल कार्यान्वयन:

- इसके लिए **FCFS क्यू** की आवश्यकता होती है, जिससे कार्यान्वयन अधिक जटिल हो जाता है।

(b) अतिरिक्त संदर्भ स्विचिंग:

- जब कोई प्रक्रिया अपने क्रिटिकल सेक्शन में प्रवेश नहीं कर पाती है:
  1. यह **रनिंग से वेटिंग स्टेट** में जाती है।
  2. बाद में, **वेटिंग से रेडी स्टेट** में जाती है।
- इन अतिरिक्त कॉन्टेक्स्ट स्विच से **ओवरहेड** उत्पन्न होता है।

क्लासिकल सिंक्रोनाइज़ेशन समस्याएं

सिंक्रोनाइज़ेशन समस्याएं जिनका समाधान सेमाफोर का उपयोग करके किया जाता है:

1. बाउंडेड बफर (उत्पादक (प्रोड्यूसर) -उपभोक्ता (कंज्यूमर)) समस्या

- एक **उत्पादक डेटा** आइटम उत्पन्न करता है जिसे उपभोक्ता द्वारा उपयोग किया जाता है।
- उत्पादक और **उपभोक्ता अतुल्यकालिक रूप** से कार्य करते हैं।
- कभी-कभी, उत्पादक उपभोक्ता की उपभोग गति से अधिक तेज़ी से **डेटा** उत्पन्न कर सकता है।
- एक सीमित बफर **उत्पादित आइटमों** को उनके उपयोग होने तक अस्थायी रूप से रखता है।
- उत्पादक और उपभोक्ता के बीच संचार साझा मेमोरी और सेमाफोर का उपयोग करके किया जाता है।

डाइनिंग फिलॉसफर प्रॉब्लम (भाग 1)

- प्रक्रिया समन्वय को दर्शाने के लिए प्रयुक्त एक क्लासिकल सिंक्रोनाइजेशन प्रॉब्लम।

विवरण:

a. एक गोलाकार मेज के चारों ओर **N दार्शनिक** बैठे हैं।

b. प्रत्येक दार्शनिक के सामने एक-एक करके N प्लेटें रखी हैं।

c. प्रत्येक जोड़ी प्लेटों के बीच में **N चॉपस्टिक** रखी हैं।

d. मेज के बीच में स्पेगेटी का एक कटोरा रखा है।

e. जब किसी दार्शनिक को भूख लगती है, तो वे अपने **पड़ोसियों** के साथ साझा की गई दो चॉपस्टिक उठाने की कोशिश करते हैं।

- यदि पड़ोसी पहले से ही खाना खा रहा हो, तो **दार्शनिक** को इंतजार करना होगा।

f. दोनों चॉपस्टिक मिलने के बाद वे **खाना** खाते हैं।

g. खाना खाने के बाद **वे दोनों चॉपस्टिक नीचे रख** देते हैं और फिर से सोचने लगते हैं।

- इसके बाद चॉपस्टिक पड़ोसियों के लिए उपलब्ध हो जाती हैं।

एल्गोरिदम की सीमा

- सभी दार्शनिकों को एक साथ भूख लग सकती है और वे सबसे पहले अपनी **बाईं चॉपस्टिक** उठा सकते हैं।
- फिर प्रत्येक व्यक्ति अपनी **दाईं चॉपस्टिक के लिए अनिश्चित काल** तक प्रतीक्षा करेगा, जिससे गतिरोध उत्पन्न हो जाएगा।
- यहां **प्रगति की आवश्यकता** पूरी नहीं होती।

डेडलॉक-फ्री सॉल्यूशन

- **एल्गोरिदम** में संशोधन करें:
  - **सम संख्या वाले दार्शनिक** (0, 2, 4, …) पहले **बाएँ, फिर दाएँ** का चयन करेंगे।
  - **विषम संख्या वाले दार्शिक** (1, 3, 5, …) पहले **दाएँ, फिर बाएँ** का चयन करेंगे।
- इससे चक्रीय प्रतीक्षा से बचा जा सकेगा, जिससे गतिरोध से मुक्ति मिलेगी।

स्लीपिंग बार्बर समस्या

समस्या का विवरण

a. एक नाई की दुकान में प्रतीक्षा क्षेत्र में **n कुर्सियाँ और नाई** के केबिन में **1 कुर्सी** है।
b. यदि **कोई ग्राहक मौजूद** नहीं है, तो नाई सो जाता है।
c. जब कोई **ग्राहक** आता है:
   - अगर नाई सो रहा होता है, तो **ग्राहक उसे जगाकर बाल** कटवा लेता है।
d. यदि कोई ग्राहक पहले से ही बाल कटवा रहा हो:
   - नया ग्राहक प्रतीक्षाकक्ष में **खाली कुर्सी** देखता है।
   - यदि कुर्सी उपलब्ध हो, तो ग्राहक बैठ जाता है और **प्रतीक्षा** करता है (पहले आओ पहले पाओ के क्रम में)।
   - यदि सभी कुर्सियाँ भरी हों, तो **ग्राहक बिना बाल** कटवाए चला जाता है।
e. जब कोई ग्राहक अपना काम पूरा कर लेता है, तो वह अगले **प्रतीक्षारत ग्राहक** को नाई के केबिन में प्रवेश करने का संकेत देता है और फिर **सैलून से बाहर** निकल जाता है।

पाठकों और लेखकों की समस्या

समस्या विवरण

a. कई सहयोगी प्रक्रियाएँ एक साझा फ़ाइल या डेटाबेस तक पहुँच रही हैं।
b. कुछ प्रक्रियाएँ केवल डेटा पढ़ती हैं — इन्हें **रीडर** कहते हैं।
c. कुछ प्रक्रियाएँ डेटा को संशोधित करती हैं — इन्हें **राइटर** कहते हैं।
d. जब कोई **राइटर लिख** रहा होता है, तो कोई अन्य रीडर या राइटर डेटा तक पहुँच नहीं सकता।
e. जब **रीडर पढ़ रहे** होते हैं, तो कई रीडर एक साथ डेटा तक पहुँच सकते हैं (किसी राइटर को अनुमति नहीं है)।

लक्ष्य:

- यह सुनिश्चित करें कि लेखकों को विशेष पहुंच प्राप्त हो और कई पाठक सुरक्षित रूप से एक साथ काम कर सकें।

प्रोग्रामिंग त्रुटि 1

पारस्परिक अपवर्जन का उल्लंघन

- गलत सेमाफोर प्रयोग का उदाहरण:

```
P1:
Signal(&S);  /* Error */
<Critical Section>
```

- यदि क्रिटिकल सेक्शन में प्रवेश करने से पहले signal() फ़ंक्शन को कॉल किया जाता है, तो **दो प्रक्रियाएँ एक साथ प्रवेश** कर सकती हैं।
- यह **पारस्परिक अपवर्जन का उल्लंघन** करता है।

प्रोग्रामिंग त्रुटि 2

सभी प्रक्रियाओं को अवरुद्ध किया जा रहा है

- एक अन्य सामान्य त्रुटि का उदाहरण:

```
P2:
Wait(&S);  /* Error */
<Critical Section>
/* signal(&S) is missing */
<Remainder Section>
```

- **signal() के अनुपस्थित** होने से अन्य सभी प्रक्रियाएं **अनिश्चित काल तक प्रतीक्षा** करती रहती हैं, जिससे प्रगति अवरुद्ध हो जाती है।

निष्कर्ष:

- **सेमाफोर** का गलत इस्तेमाल **गतिरोध या पारस्परिक अपवर्जन** विफलता का कारण बन सकता है।

मॉनिटर

- **मॉनिटर एक उच्च-स्तरीय सिंक्रोनाइज़ेशन संरचना है जो सेमाफोर की सीमाओं को दूर करती है।**
- सेमाफोर में, ऑपरेशन (प्रतीक्षा, संकेत) कठोर और त्रुटि-प्रवण होते हैं।
- **मॉनिटर निम्नलिखित को समाहित करता है:**
  - वैश्विक चर प्रक्रियाओं के बीच साझा किए जाते हैं।
  - **प्रोसेस/फ़ंक्शन** जो इन चरों तक पहुँच प्राप्त करते हैं।

प्रमुख विशेषताऐं:

- **मॉनिटर** के अंदर मौजूद ग्लोबल वेरिएबल्स को केवल उसी के प्रोसीजर द्वारा एक्सेस किया जा सकता है।
- ऑपरेटिंग सिस्टम यह सुनिश्चित करता है:
  - **म्यूचुअल एक्सक्लूजन**
  - **बाउंडेड वेटिंग**
- किसी भी समय **मॉनिटर** के अंदर केवल एक ही प्रोसेस सक्रिय हो सकती है।

सेमाफोर का सारांश

अवधारणा	**विवरण / मुख्य बिंदु**
सेमाफोर	साझा संसाधनों तक पहुंच को नियंत्रित करने वाला सिंक्रोनाइज़ेशन टूल।
बाइनरी सेमाफोर	मान 0/1; सरल है लेकिन व्यस्त प्रतीक्षा का कारण बनता है; इसे स्पिनलॉक कहा जाता है।
काउंटिंग सेमाफोर	संसाधनों के कई इंस्टेंस की अनुमति देता है; व्यस्त प्रतीक्षा से बचाता है।
सेमाफोर की गणना के लाभ	FCFS एक्सेस, सीमित प्रतीक्षा अवधि, CPU का कोई अपव्यय नहीं।
हानि	जटिल कार्यान्वयन; अतिरिक्त संदर्भ स्विच।
क्लासिकल समस्याएँ	बाउंडेड बफर, डाइनिंग फिलॉसफर, स्लीपिंग बार्बर, रीडर्स-राइटर्स।
सामान्य त्रुटियों	गलत तरीके से उपयोग किए गए wait() signal() के कारण डेडलॉक या एक्सक्लूजन विफलता हो सकती है।
मॉनिटर	उच्च स्तरीय सिंक्रोनाइज़ेशन टूल जो स्वचालित पारस्परिक अपवर्जन सुनिश्चित करता है।



PW Web/App: https://smart.link/7wwosivoicgd4

Library- https://smart.link/sdfez8ejd80if